

ICG Online Assignment (C1)

Implementing C goto

Time: 40 Mins Total Marks: 10

Problem

In this online, you are required to extend the grammar to support **goto statement** as in C programming language. The goto statement is used to jump directly from one part of a program to another using a label. A goto label is an identifier (ID) followed by colon (:). A label does not conflict with a variable with the same name. The destination can be before or after the goto. Thus, your program should support the following syntax.

```
int print_val;  
print_val = 10;  
goto print_val; // jump to print_val label  
print_val = 20;  
print_val:  
    println(print_val); // this will print 10  
    goto print_val; // jump to print_val label
```

Task

1. Extend the current grammar to support **goto statement**. There will be no syntax or semantic errors in the given input program. Additionally, in the input program, there will be no other statement in the same line containing a goto label.
2. Update your ICG code to generate assembly code for this new rule/syntax. The assembly code output by your program must correctly be assembled with 'fasm'. Subsequently, the binary produced by 'fasm' from your assembly code must produce the expected output for the input program.

Mark Distribution

- Extending the grammar to support goto statement. (5)
- Code generation for goto statement. (5)

Find a sample input-output and some helper code in the next pages.

Sample Input-Output

Input Program	Generated Sample Assembly (print library omitted)
<pre> 1 int a, b; 2 int main(){ 3 int x, y; 4 a = 10; 5 b = 20; 6 x = 100; 7 y = 200; 8 println(a); 9 goto L1; 10 L2: 11 println(b); 12 goto L3; 13 L1: 14 println(x); 15 goto L2; 16 L3: 17 println(y); 18 return 0; 19 } 20 </pre>	<pre> format ELF executable 3 entry main segment readable writeable a dd 1 DUP (0) b dd 1 DUP (0) segment readable executable main: PUSH EBP MOV EBP, ESP SUB ESP, 4 SUB ESP, 4 .L1: MOV EAX, 10 ; Line 4 MOV [a], EAX .L2: MOV EAX, 20 ; Line 5 MOV [b], EAX .L3: MOV EAX, 100 ; Line 6 MOV [EBP-4], EAX .L4: MOV EAX, 200 ; Line 7 MOV [EBP-8], EAX .L5: MOV EAX, [a] ; Line 8 CALL print_number .L6: JMP .L7 .L8: .L9: .L10: MOV EAX, [b] ; Line 11 CALL print_number .L11: JMP .L12 .L13: .L7: .L14: MOV EAX, [EBP-4] ; Line 14 CALL print_number .L15: JMP .L9 .L16: .L12: .L17: MOV EAX, [EBP-8] ; Line 17 CALL print_number .L18: MOV EAX, 0 ; Line 18 </pre>
Expected Output (from the generated binary by fasm)	
<pre> 10 100 20 200 </pre>	

	<pre> JMP .L20 .L19: .L20: ADD ESP, 8 POP EBP MOV EAX, 1 XOR EBX, EBX INT 0x80 POP EBP RET </pre>
--	---

Helper Code

- If you want to use a map, a sample code on how to use a map in C++ is given below.

```

#include <iostream>
#include <map>
using namespace std;
int main() {
    map<string, string> dict;
    // Store key-value pairs in the map
    dict["foo"] = "bar";
    //search value against a key in the map
    auto it = dict.find("foo");
    if (it != dict.end()) //key in found
        cout << "Value = " << it->second << endl;
    else cout << "Not found" << endl;
    return 0;
}

```